

北京理工大学

本科生毕业设计（论文）

游戏化前端学习移动 APP 的设计与实现

Design and Implementation of a Gamified Front-End Learning
Mobile App

学 院： -

专 业： -

班 级： -

学生姓名： -

学 号： -

指导教师： -

2026 年 5 月 12 日

原创性声明

特此申明。

本人签名: _____ 日期: _____ 年 _____ 月 _____ 日

关于使用授权的声明

本人签名: _____ 日期: _____ 年 _____ 月 _____ 日

指导老师签名: _____ 日期: _____ 年 _____ 月 _____ 日

游戏化前端学习移动 APP 的设计与实现

摘 要

本文围绕一款面向前端初学者的游戏化学习移动应用的设计与实现展开研究。针对传统前端入门学习中学习动机不足、练习反馈不及时以及移动场景适配不足等问题，论文结合游戏化学习与移动学习的相关理论，设计并实现了一个以移动客户端为核心、服务端与管理后台端为支撑的学习系统。该系统围绕课程学习、章节阅读、编码练习、闯关挑战、每日挑战、社交互动与个人成长反馈等功能构建学习闭环，并通过奖励机制、排行榜和任务节奏增强学习过程的参与感与持续性。

在系统实现上，移动客户端采用 Flutter 进行跨平台开发，结合 GetX 实现路由管理、状态管理与依赖注入，使用 Dio 与 GetStorage 分别完成接口访问与本地持久化；服务端采用 Rust、Salvo 与 PostgreSQL 提供统一业务支撑；同时以 OpenAPI 文档作为三端共享的接口约束基础，用于保证接口结构与联调过程的一致性。论文进一步从需求分析、总体设计、关键功能实现与功能验证四个层面对系统进行说明。

相关实现与验证情况说明，该系统在当前实现范围内能够完成注册登录、课程学习、练习提交、挑战参与、奖励反馈、排行榜展示与基础联调验证等核心流程，能够支撑移动学习闭环的基本运行。本文的工作可为面向前端基础教育场景的移动学习应用设计与实现提供一定参考。

关键词：游戏化学习；移动学习；前端教育；Flutter；学习应用

Design and Implementation of a Gamified Front-End Learning Mobile App

Abstract

This thesis studies the design and implementation of a gamified mobile learning application for front-end beginners. To address the problems of weak learning motivation, delayed practice feedback, and insufficient support for mobile learning scenarios in traditional introductory front-end learning, the work combines theories of gamified learning and mobile learning to design and implement a learning system with the mobile application as the core and the server side together with the admin dashboard as supporting components. The system forms a learning loop through course learning, chapter reading, coding practice, level-based challenges, daily challenges, social interaction, and growth feedback, while using rewards, rankings, and task rhythm to strengthen engagement and persistence.

On the implementation side, the learner client is developed with Flutter for cross-platform delivery, uses GetX for routing, state management, and dependency injection, and adopts Dio and GetStorage for API access and local persistence. The server side is implemented with Rust, Salvo, and PostgreSQL to provide unified business support. Meanwhile, an OpenAPI document is used as the shared contract among the mobile client, server side, and admin dashboard to ensure interface consistency during development and integration. The thesis further explains the system from four aspects: requirements analysis, overall design, key function implementation, and functional verification.

The results show that, within the current implementation scope, the system can complete core workflows such as registration and login, course learning, exercise submission, challenge participation, reward feedback, leaderboard display, and basic cross-end integration verification. It therefore demonstrates a relatively complete mobile learning loop and multi-end collaboration capability. The work can provide a useful reference for the design and implementation of mobile learning applications oriented to introductory front-end education.

Key Words: gamified learning; mobile learning; front-end education; Flutter; learning application

目 录

摘 要	I
Abstract	II
第 1 章 绪论	1
1.1 研究背景	1
1.2 研究目的与意义	1
1.3 国内外研究现状	2
1.4 研究内容与论文结构	3
第 2 章 相关理论与关键技术	6
2.1 游戏化学习相关理论	6
2.2 移动学习与智能教育支持	6
2.3 前端学习与编程教学相关支撑	7
2.4 系统实现相关技术	8
第 3 章 需求分析	10
3.1 总体需求分析	10
3.2 技术需求分析	11
3.3 功能需求分析	11
3.4 非功能需求分析	12
第 4 章 系统设计	15
4.1 总体架构设计	15
4.2 功能结构设计	16
4.3 核心业务流程设计	18
4.4 接口与数据设计	19
4.5 服务端与管理后台端协同设计	20
第 5 章 系统实现与测试	23
5.1 移动客户端关键功能实现	23
5.2 服务端与管理后台端关键实现	26
5.3 功能测试与联调验证	27
5.4 测试结果分析	29
结 论	32
参考文献	33

附 录	34
附录 A AI 三级智能提示设计与 Prompt 示例	34
附录 B OpenAPI 接口列表与对应功能表单	37
致 谢	41

第 1 章 绪论

1.1 研究背景

随着移动互联网终端的普及与碎片化学习方式的发展，基于智能手机开展知识学习已经成为常见的学习场景之一。对于前端基础知识学习而言，HTML 与 CSS 等内容既具有较强的实践性，又要求学习者在理解概念的同时进行持续练习，因此传统以被动阅读为主的学习方式往往难以长期保持学习动机。将积分、徽章、关卡、排行榜等机制引入学习过程，能够在一定程度上增强学习者的参与感、反馈感与阶段性成就感，从而提升学习持续性与主动性^[1-3]。

与此同时，移动学习强调学习过程与时间、地点的解耦，使学习者能够在通勤、课余等碎片时间中完成短时高频的学习活动。对于前端入门教育而言，若能够结合课程阅读、编码练习、闯关挑战、每日任务与社交反馈等机制，构建面向移动客户端的完整学习闭环，则有助于提升学习过程的连贯性和趣味性，并为前端基础能力培养提供更具交互性的支持环境^[4-5]。

前端技术作为 Web 开发的基础，其学习需求持续增长。HTML 负责页面结构，CSS 负责页面样式，两者共同构成了 Web 页面的基础。对于初学者而言，这些技术虽然入门门槛相对较低，但要达到熟练运用水平仍需要大量实践。传统的学习方式主要依赖教材阅读和静态示例观摩，学习者难以获得及时的练习反馈和进度确认。而游戏化学习应用通过将学习内容组织为任务链、将练习过程设计为挑战关卡、将学习成果可视化为成长数据，能够在一定程度上弥补传统学习方式的不足，为前端基础教育提供更具吸引力的学习载体。

1.2 研究目的与意义

本文面向前端初学者，围绕一款游戏化前端学习移动应用的设计与实现展开研究，目标是构建一个以移动客户端为核心的学习系统，使学习者能够在课程学习、代码练习、挑战通关、每日任务与社交激励的协同作用下完成前端基础知识的持续学习。系统在功能上强调学习闭环的完整性，在实现上强调移动客户端体验与服务端、管理后台端协同的一致性。

从实践意义上看，该系统尝试将游戏化学习机制与移动学习场景结合，探索更适

合前端基础教育的应用实现方式。游戏化机制在教育领域的应用已有较多研究，但面向前端技术学习、以移动应用为载体的系统化实现案例仍然较少。本系统通过将课程学习、编码练习、挑战机制和社交反馈整合到移动客户端中，为前端初学者提供了一种新的学习途径，其实践价值在于验证这种整合方式的可行性，并为后续效果评估提供实现基础。

从工程意义上看，本文通过移动客户端、服务端、管理后台端与契约文档之间的协同设计，形成一套可落地的多端联动实现方案，为同类学习应用的设计与开发提供参考^[6-8]。在工程实现层面，系统涉及移动应用开发、服务端构建、管理界面开发和接口契约维护等多个技术领域，这些领域之间的协同工作是系统成功落地的关键。本文在实现过程中积累的多端协作经验和选型思考，对于其他需要类似架构的学习应用具有一定的参考价值。

1.3 国内外研究现状

现有研究表明，游戏化通过在非游戏环境中引入游戏设计元素，可以对学习动机、学习投入与持续使用行为产生积极影响，但其具体效果仍与学习情境、任务设计与用户特征密切相关。相关研究还指出，教育游戏、游戏化学习与移动学习在目标、方法与应用侧重点上存在差异，研究者通常需要结合具体学习对象与学习目标进行设计，而不能简单依赖单一游戏元素获得稳定效果^[1-2,8]。

Deterding 等人对游戏化的定义和构成要素进行了系统梳理，为后续研究提供了概念基础。Hamari 等人通过文献综述发现，游戏化在教育领域的应用效果存在较大差异，其成功依赖于游戏元素与学习目标的匹配程度。Kalmpourtzis 从教育游戏设计角度指出，有效的游戏化设计需要深入理解学习者的动机结构和认知特征，而非简单套用游戏机制。这些研究为本系统的设计提供了理论指导，使系统在游戏化机制的选择上更加注重与学习内容的契合度。

在智能教育与学习支持系统领域，已有研究围绕智能辅导系统、学习分析、自适应教育系统以及教育数据挖掘等方向形成了较为丰富的理论与实践积累，这些研究为学习过程支持、学习行为分析和个性化干预提供了重要思路^[9-12]。

智能辅导系统研究关注如何通过计算机程序模拟人类教师的一对一辅导过程，为学习者提供个性化的学习指导。Koedinger 等人的研究表明，智能辅导系统能够有效提升学习效果，但其开发成本和技术复杂度较高。自适应教育系统研究关注如何

根据学习者的实时表现动态调整学习内容和路径，Brusilovsky 等人对用户建模和自适应策略进行了系统总结。学习分析技术研究关注如何从学习行为数据中提取有价值的信息，Papamitsiou 等人的综述显示了该领域从理论探索向实践应用的转变趋势。本系统虽然未实现完整的智能辅导或自适应推荐功能，但在进度记录、错误提示和成就反馈等方面借鉴了这些研究的思路。

在编程教育与 Web 开发教学领域，研究重点主要集中在编程入门难点、教学支撑工具、Web 开发学习方式以及程序可视化辅助等方面。相关研究说明，编程学习既依赖知识讲解，也依赖可操作、可反馈、可重复的实践过程，而这与游戏化学习应用强调任务驱动和即时反馈的设计逻辑具有一定一致性^[13-15]。

Robins 等人对编程教学中的困难进行了系统分析，指出概念理解、错误定位和知识迁移是初学者面临的主要挑战。Heines 的研究表明，Web 开发教学需要特别关注实践环节的设计，使学习者能够在真实或接近真实的环境中进行练习。Guo 开发的 Python Tutor 工具展示了程序可视化在编程教育中的价值，通过图形化展示程序执行过程帮助学习者理解代码行为。这些研究为本系统的练习设计和反馈机制提供了参考，使系统在设计编码练习时更加注重错误提示的可理解性和学习过程的可视化。

总体来看，现有研究为游戏化学习、智能教育支持与编程教学提供了理论基础，但面向前端基础教育场景、以移动应用为主体、同时结合课程学习、代码练习、挑战机制与社交反馈的系统化实现案例仍具有较强的实践研究价值。

1.4 研究内容与论文结构

本文以游戏化前端学习移动应用为研究对象，对系统的需求分析、总体设计、关键功能实现与功能验证过程进行梳理和总结。论文内容主要包括相关理论与关键技术、系统需求分析、系统设计、系统实现与测试等部分。其中，移动客户端学习应用为全文叙事主体，服务端与管理后台端作为支撑系统，OpenAPI 契约文档作为协同约束依据，共同用于保证学习流程闭环与系统协同实现。

全文共分为五章。第一章为绪论，介绍研究背景、研究意义、研究现状与论文结构；第二章为相关理论与关键技术，说明游戏化学习、移动学习及系统开发所涉及的关键技术；第三章为需求分析，梳理系统的总体需求、功能需求与非功能需求；第四章为系统设计，说明系统总体架构、功能结构、业务流程与接口协同方式；第五章为系统实现与测试，介绍移动客户端关键功能与服务端、管理后台端关键实现，

并对系统功能验证与三端联调情况进行说明。

在论文的叙事逻辑上，本文始终围绕移动客户端学习体验展开，服务端和管理后台端作为支撑系统配合说明。这种叙事方式既保证了论文焦点的集中性，又体现了系统实现的多端协同特征。各章节之间保持逻辑递进关系，从理论基础到需求分析，从系统设计到实现测试，逐步深入，使读者能够系统了解本游戏化前端学习应用的全貌。

需要说明的是，本文的研究范围限定在系统的设计与实现层面，不涉及教学效果实验、用户行为分析或商业化运营策略等后续环节。游戏化机制对学习动机和持续学习行为的影响虽然已有相关理论基础，但其在具体系统中的表现仍需结合实际使用数据进行验证。本文通过构建可运行的系统原型，为这类后续验证工作提供了技术基础，但本身并不声称已经证明了游戏化学习在特定场景下的效果优势。这种范围界定既符合本科毕业设计的实际条件，也使论文能够聚焦于工程实现层面的深入分析。

在研究方法上，本文采用工程实践与文献分析相结合的方式进行。工程实践方面，通过实际的系统开发过程验证设计方案的可行性，在开发过程中不断调整和完善设计决策。文献分析方面，通过查阅游戏化学习、移动教育和编程教学等领域的相关研究，为系统设计提供理论支撑和参考借鉴。这种理论与实践相结合的方法使系统既有理论依据，又具备实际可行性，避免了纯理论研究的空洞和纯工程实践的盲目。

在创新点方面，本文的主要贡献不在于提出全新的游戏化理论或突破性的技术方案，而在于将已有的游戏化学习理念与前端基础教育场景相结合，通过移动客户端、服务端和管理后台端的协同实现，构建了一个可运行的学习应用原型。这一贡献的价值在于探索了游戏化机制在前端技术学习领域的具体应用方式，以及多端协同架构在移动学习场景中的实现路径，为后续研究和实践提供了参考案例和基础平台。

从应用场景来看，前端技术学习具有知识更新快、实践性强和学习者基数大的特点。传统的课堂教学和在线教程虽然能够覆盖知识传授环节，但在学习动机维持、实践反馈及时性和学习进度可视化等方面存在不足。本系统尝试通过移动客户端将学习过程游戏化，使学习者能够在碎片化时间中完成短时高频的学习活动，并通过积分、徽章、排行榜等机制获得即时的正向反馈。这种设计思路对于其他需要大量

实践练习的技术学习领域也具有一定的借鉴意义，例如后端开发、移动应用开发和数据科学等方向的基础教育。

第2章 相关理论与关键技术

2.1 游戏化学习相关理论

游戏化学习强调在非游戏情境中引入积分、徽章、等级、任务、反馈与排行榜等游戏设计元素，以提高学习活动的参与度和持续性。与完整的教育游戏相比，游戏化更关注将游戏机制嵌入既有学习任务之中，使学习者在完成真实学习目标的过程中获得更强的行为激励与过程反馈。对于前端基础知识学习而言，课程阅读、代码练习与阶段性挑战天然具有任务化特征，因此较适合与游戏化机制进行结合^[1-2,7]。

从学习动机角度看，游戏化机制能够通过即时反馈、阶段奖励和目标可视化增强学习者的胜任感与进度感。当学习者完成一道练习或通过一个关卡时，系统立即给予积分、徽章或等级提升等反馈，这种即时正向强化有助于建立学习与奖励之间的关联，增强学习行为的持续性。对初学者而言，这种设计有助于缓解前端入门阶段中因知识抽象、练习反复与错误频发所带来的挫败感，提高其继续学习的意愿。因此，在本系统中，积分奖励、关卡挑战、每日任务和排行榜并不是附加装饰，而是服务于学习行为持续发生的交互机制。这一思路与自我决定理论中关于自主性、胜任感和归属感的解释相一致^[3,8]。

自我决定理论认为，人类有三种基本心理需求：自主性需求、胜任感需求和归属感需求。在游戏化学习环境中，自主性需求体现为学习者可以选择学习路径、挑战难度和学习节奏；胜任感需求体现为通过任务完成和技能提升获得的能力确认；归属感需求体现为通过排行榜、好友互动等社交机制获得的群体认同。本系统在设计时考虑了这三种需求的平衡，既提供结构化的课程路径保证学习系统性，又通过挑战选择和社交功能给予学习者一定的自主空间。

2.2 移动学习与智能教育支持

移动学习强调学习者能够借助移动终端在不同时间、不同地点完成学习活动，其核心特征包括学习场景灵活、学习过程碎片化、交互方式轻量化以及设备依赖性明显。与桌面环境相比，移动客户端在屏幕空间、输入方式、网络稳定性与操作节奏方面存在天然限制，因此移动学习应用在设计时需要更加重视信息层级、任务流程、反馈节奏与离线容错能力^[4-5]。

移动学习的碎片化特征意味着学习者通常只能在较短时间内集中注意力，单次学习时长可能只有几分钟到十几分钟。这种特征对学习内容的设计提出了特殊要求：内容单元应足够紧凑，使学习者能够在一次碎片时间内完成一个完整的学习任务；进度保存应足够及时，使学习者可以随时中断并在下次继续；反馈应足够迅速，使学习者能够在有限时间内获得学习效果的确证。本系统的章节设计、练习长度和每日挑战难度都考虑了这种碎片化特征，力求使每个学习单元都能在典型碎片时间内完成。

在教育技术发展过程中，智能辅导系统、自适应教育系统与学习分析方法不断丰富，为学习资源推荐、学习路径调整、学习行为记录与反馈设计提供了重要参考。虽然本文并不以构建复杂智能教学系统为核心目标，但这些研究为系统中的辅助提示、进度记录和成长反馈设计提供了理论借鉴^[9-12]。

智能辅导系统的核心思想是通过建模学习者的知识状态，提供个性化的教学指导和练习推荐。自适应教育系统则进一步关注学习路径的动态调整，根据学习者的表现实时改变后续内容的难度和类型。学习分析技术通过收集和处理学习行为数据，为教学决策和系统优化提供数据支持。本系统虽然未实现完整的知识追踪模型或自适应推荐算法，但在进度记录、错误提示和成就反馈等方面借鉴了这些研究的思路，为后续功能扩展预留了空间。

2.3 前端学习与编程教学相关支撑

前端基础学习兼具知识理解与实践操作双重特征，因此其教学过程既需要结构化内容呈现，也需要可反馈的编码练习环境。编程教育研究普遍认为，程序设计教学中的困难主要体现在抽象概念理解、错误定位、实践反馈以及学习迁移等方面。初学者在学习 HTML 和 CSS 时，常常难以理解抽象的标签语义和样式规则，更难将所学知识应用到新的页面布局场景中。与此相应，Web 开发教学与程序可视化工具研究说明，良好的交互反馈和过程可视化能够降低学习门槛，提高学习者的理解效率和练习积极性^[13-15]。

在错误定位方面，前端代码的错误往往不会导致程序崩溃，而是表现为页面显示异常，这种静默失败特征增加了初学者定位问题的难度。因此，系统在设计练习反馈时，不仅返回测试用例是否通过的结果，还尽可能提供具体的错误描述和修正建议，帮助学习者理解问题所在。在学习迁移方面，系统通过不同难度的挑战任务

和多样化的练习场景，促使学习者将基础知识应用到不同的实际问题中，逐步培养灵活运用能力。

本系统以 HTML 与 CSS 为前端学习切入点，通过课程内容、代码练习、挑战关卡和成长反馈相结合的方式组织学习流程，其设计思路与上述研究在实践训练和即时反馈方面具有一致性。HTML 作为页面结构描述语言，其学习重点在于理解各种标签的语义和适用场景；CSS 作为页面样式描述语言，其学习重点在于理解选择器、盒模型、布局方式等核心概念。这两部分内容既相互独立又紧密关联，系统通过课程组织使学习者先分别掌握基础知识，再通过综合练习和挑战将两者结合运用。

2.4 系统实现相关技术

从工程实现角度看，学习应用的构建不仅依赖教育理论，也依赖稳定的软件工程方法和适合的技术实现路径。软件工程理论为需求梳理、系统分层、模块协作与后续维护提供了方法基础，而游戏设计与教育游戏设计相关研究则为任务组织、反馈节奏和学习体验构建提供了实践参考^[6-8,16]。

软件工程方法强调通过需求分析、系统设计、实现和测试等阶段有序推进项目，在每个阶段建立明确的交付物和验证标准。本系统的开发过程遵循这一基本流程，在需求分析阶段明确功能范围和非功能约束，在系统设计阶段确定架构方案和技术选型，在实现阶段按模块逐步开发，在测试阶段验证功能正确性和接口一致性。这种分阶段推进方式有助于控制项目风险，使问题能够在早期被发现和解决。

本文在技术实现上采用移动客户端、服务端与管理后台端协同的方式完成系统搭建。移动客户端选择 Flutter 框架进行跨平台开发，使同一套代码能够同时构建 iOS 和 Android 应用，降低多端维护成本。服务端选择 Rust 语言和 Salvo 框架，利用 Rust 的内存安全特性和高性能特征，为移动客户端提供稳定可靠的接口服务。管理后台端选择 Vue.js 框架，借助其组件化开发模式和丰富的生态，快速构建运营人员使用的管理界面。

相关技术选择服务于系统工程落地，而非作为论文创新点单独展开。其核心价值在于支撑课程学习、编码练习、挑战反馈与多端联动的整体实现。在技术选型时，优先考虑了团队熟悉度、社区活跃度和文档完善度等因素，使技术风险控制在可接受范围内。同时，技术栈的确定也考虑了后续维护和功能扩展的需求，选择了具有良好生态和持续发展潜力的技术方案。

Flutter 框架采用 Dart 语言开发，通过自绘渲染引擎实现跨平台一致性，其热重载特性能够显著提升开发迭代效率。在移动客户端开发中，Flutter 的 Widget 系统提供了丰富的界面构建能力，使开发者能够以声明式方式描述界面结构和交互行为。GetX 作为 Flutter 的状态管理方案，提供了依赖注入、路由管理和状态响应式更新等能力，其轻量级特性适合中小型项目的快速开发。

Rust 语言以其内存安全保证和零成本抽象著称，适合构建需要高可靠性的服务端应用。Salvo 作为 Rust 生态中的 Web 框架，提供了异步请求处理、中间件支持和路由组织等基础能力，其设计理念与 Rust 语言特性相契合，能够在保证性能的同时提供较好的开发体验。在服务端开发中，异步编程模型能够有效处理并发请求，而 Rust 的所有权系统则在编译期避免了常见的内存安全问题。

Vue.js 作为渐进式前端框架，其组件化开发模式和响应式数据绑定机制使界面开发更加高效。在管理后台端开发中，Vue.js 的单文件组件结构使模板、脚本和样式能够集中管理，配合 Element Plus 等组件库可以快速构建功能完善的管理界面。Axios 作为 HTTP 客户端库，提供了请求拦截、响应处理和错误管理等能力，与服务端接口的对接过程较为顺畅。

在接口契约方面，系统采用 OpenAPI 规范描述接口定义，使三端能够在统一的文档基础上进行开发和联调。OpenAPI 文档作为三端共享契约，并可用于导出接口文档页面，提高开发沟通的一致性。在数据持久化方面，系统采用关系型数据库存储业务数据，通过合理的表结构设计和索引优化保证基础查询性能。在数据迁移方面，系统使用迁移脚本管理数据库模式变更，使开发和生产环境的数据库结构能够保持一致，便于团队协作和版本控制。

综合来看，本系统的技术选型遵循了实用性、可维护性和可扩展性原则，各技术组件之间形成了较为合理的分工协作关系。移动客户端负责用户交互和状态管理，服务端负责业务逻辑和数据处理，管理后台端负责内容运营和系统配置，接口契约负责多端协同的标准化。这种技术架构既满足了当前功能需求，也为后续功能扩展和性能优化预留了空间。

第3章 需求分析

3.1 总体需求分析

本系统面向前端初学者构建移动客户端学习环境，目标是在移动设备上形成课程学习、代码练习、闯关挑战、每日任务、社交激励与个人成长相结合的完整学习闭环。根据需求文档，系统由移动客户端、服务端与管理后台端组成，其中移动客户端承担主要学习活动，服务端负责统一业务处理与数据支撑，管理后台端负责内容维护、运营支撑与数据管理。该类系统既需要满足移动学习场景下的便捷访问需求，也需要兼顾编程教育中实践反馈及时、任务组织明确和学习持续性较强等特点^[6,13-14]。

从学习目标上看，系统不仅要支持前端基础知识的呈现和阅读，还应支持学习者通过编码练习和挑战任务完成知识内化，并借助成长反馈与社交激励增强持续学习意愿。因此，系统需求不能只停留在内容展示层面，而需要覆盖学习行为记录、任务组织、结果反馈和成长激励等多个方面^[2-3,8]。前端基础知识包括 HTML 标记语言、CSS 样式语言和基础页面布局等内容，这些知识既具有明确的语法规则，又需要通过大量实践才能熟练掌握，因此系统需求特别强调理论与实践的结合。

从用户群体特征来看，目标用户主要是前端初学者，他们可能具备一定的计算机使用经验，但对 Web 开发技术尚不熟悉。这类用户在学习过程中容易因概念抽象、错误难以定位或缺乏及时反馈而产生挫败感，因此系统需要在交互设计上注重引导性、反馈的及时性和错误提示的可理解性。同时，初学者通常没有固定的学习时间和场所，移动学习场景正好契合其利用碎片时间学习的需求，但也对系统的离线能力和状态恢复能力提出了更高要求。

从系统边界来看，本文聚焦于学习应用本身的设计与实现，不包括完整的在线代码执行环境、大规模用户行为分析系统或商业化运营平台。AI 辅助功能限定在错误解释和提示层面，不扩展为完整的智能辅导系统。社交功能限定在好友、动态和排行榜等基础互动形式，不扩展为实时通信或协作编程等复杂场景。这些边界界定使系统需求保持在一个可实现的范围内，同时为核心学习闭环的完整性提供保障。

3.2 技术需求分析

在运行平台方面，移动客户端需要支持 iOS 与 Android 智能手机，适配不同尺寸屏幕并保持基础交互一致性；管理后台端主要面向桌面浏览器环境，用于内容维护与配置操作；服务端需要具备面向多端的统一 API 提供能力。由于系统存在移动客户端学习场景，因此网络环境不能假设始终稳定，应用需要具备一定的离线感知与本地缓存能力^[4-5]。

移动客户端的技术需求还包括对移动设备特性的适配。例如，需要处理不同屏幕尺寸和分辨率的布局适配问题，需要在虚拟键盘弹出时调整界面元素位置，需要处理应用前后台切换时的状态保存和恢复。在性能方面，移动客户端需要控制内存占用和电量消耗，避免长时间使用导致设备过热或电量快速下降。在存储方面，移动客户端需要合理利用本地存储空间，对缓存数据进行有效管理，避免因存储膨胀影响应用性能。

在系统协同方面，服务端接口必须能够同时支撑移动客户端与管理后台端的访问需求，并通过统一响应结构保障调用一致性。与此同时，接口文档需要作为开发与联调的共同依据，以减少多端协作中的理解偏差。对移动客户端而言，还需要具备认证信息持久化、进度缓存、弱网回退与错误处理能力，以保证学习流程的连续性^[6,11-12]。

服务端的技术需求还包括对并发访问的处理能力、对数据库操作的事务保障、对异常情况的容错处理等。虽然本文不涉及大规模性能优化，但服务端需要保证在正常负载下能够稳定响应请求，在异常情况下能够给出合理的错误信息而不是直接崩溃。管理后台端的技术需求则侧重于内容编辑的便利性、数据展示的清晰性和操作反馈的及时性，使运营人员能够高效地完成日常维护工作。

3.3 功能需求分析

结合需求文档与现有实现，移动客户端功能可归纳为以下几个方面。第一是账户管理功能，包括注册、登录与个人资料维护，用于建立用户身份与学习档案。第二是课程学习功能，包括课程列表浏览、课程详情查看、章节阅读和学习进度跟踪，用于支撑基础知识学习过程。第三是编码练习功能，包括练习详情查看、代码提交与结果反馈，用于连接知识理解与实践操作^[13-15]。

第四是挑战相关功能，包括闯关挑战、星级评价、奖励发放与每日挑战，用于

将学习活动组织为具有目标感与节奏感的任务链。第五是 AI 辅助功能，用于在学习者练习受阻时提供错误解释与必要提示，但其范围应限定在辅助解释层面，而不扩展为完整的代码生成或个性化推荐系统。第六是社交互动功能，包括好友、动态与排行榜，用于提供外部激励与学习氛围。第七是成长反馈功能，包括经验值、徽章、等级与统计信息，用于沉淀学习成果并强化用户成长感知^[4-5,9-10]。

在账户管理功能中，注册流程需要收集用户基础信息并验证输入有效性，登录流程需要验证身份并发放认证凭证，个人资料维护需要支持昵称、头像等信息的修改。这些功能虽然基础，但直接影响用户首次使用体验和后续身份识别，因此需要在易用性和安全性之间取得平衡。

在课程学习功能中，课程列表需要支持分类筛选和搜索，课程详情需要展示完整的课程信息和章节目录，章节阅读需要提供舒适的内容展示和进度记录。学习进度跟踪需要记录每门课程、每个章节的学习状态，使学习者能够随时了解自己的学习进展，也为系统的个性化推荐和成长反馈提供数据基础。

在编码练习功能中，题目展示需要清晰说明要求和示例，代码编辑需要提供基础的输入环境，结果反馈需要及时返回编译和测试结果。这些功能的实现质量直接影响学习者的练习体验，特别是错误反馈的可理解性，对于帮助初学者定位和修正错误至关重要。

对于支撑系统而言，管理后台端需具备课程、题目、挑战、用户与公告等内容管理能力，以支持移动客户端的内容供给与日常运营；服务端则需提供统一的认证、内容访问、提交记录、奖励结算、进度管理与社交数据接口，从而保证学习闭环能够在多端环境中稳定运行^[6,16]。

3.4 非功能需求分析

在可用性方面，系统应保证学习者能够以较短路径完成主要学习任务，界面反馈应清晰，状态切换应明确，减少因复杂操作造成的学习中断。具体而言，从打开应用到进入课程学习的操作步骤应控制在三步以内，常用功能应在主界面直接可达，避免深层嵌套的导航结构。错误提示应使用学习者能够理解的语言，避免展示技术性错误代码或堆栈信息^[14-15]。

在响应性方面，系统需要保证课程加载、页面切换、请求返回与错误反馈具备基本可接受的交互速度，并能在网络异常时给出可理解的提示。课程列表和章节内

容的加载时间应控制在用户可接受的范围内，代码提交后的结果反馈应尽可能及时，避免学习者因等待时间过长而产生焦虑。在网络异常时，系统应明确告知用户当前网络状态，并提供重试或稍后操作的选项，而不是无限等待或显示空白页面。

在可维护性方面，移动客户端、服务端与管理后台端应保持职责清晰，接口结构统一，便于后续功能扩展与问题定位。代码组织应遵循模块化原则，各功能模块之间的耦合度应控制在合理范围内，使单个模块的修改不会对其他模块产生意外影响。文档应覆盖关键设计决策和接口定义，使新加入的开发者能够快速理解系统结构。

在一致性方面，系统需要通过统一契约文档控制接口字段、响应包络与权限边界，保证多端协作不因接口漂移而失效。移动客户端的界面风格应保持一致，相同功能的交互方式应在不同页面中遵循相同模式，减少学习者的认知负担。服务端对同类请求的处理逻辑应保持一致，避免因特殊处理导致的意外行为。

在可靠性方面，移动客户端应支持本地存储和离线回退策略，使学习记录不会因为短时断网而直接丢失。关键操作如代码提交应在本地保存副本，网络恢复后自动重试提交，避免因网络抖动导致学习成果丢失。服务端应保证数据持久化的可靠性，对关键操作采用事务保障，避免部分更新导致的数据不一致。

在扩展性方面，系统需要为后续课程扩充、挑战增加和能力增强保留一定结构空间，但本文只围绕当前实现范围展开，不对后续版本功能进行详细设计^[6,12]。具体而言，课程和挑战的数据模型应支持新增字段而不影响现有功能，接口设计应预留版本控制机制以便后续演进，功能模块的划分应使新增功能能够以插件方式接入而不需要大规模重构现有代码。

在兼容性方面，移动客户端需要适配主流 iOS 和 Android 系统版本，在保证核心功能可用的前提下，对较旧版本系统提供降级体验。服务端接口设计应遵循向后兼容原则，避免因接口变更导致旧版本移动客户端无法正常使用。管理后台端需要兼容主流桌面浏览器，确保运营人员能够在常见浏览器环境中顺利完成内容管理工作。

在可访问性方面，系统应遵循基本的可访问性设计原则，包括界面元素具有足够的对比度、交互控件具有适当的尺寸、状态变化具有明确的视觉反馈等。虽然本文不将可访问性作为核心研究目标，但这些基础考虑有助于扩大系统的适用人群，使更多学习者能够顺利使用系统进行前端学习。

在数据安全与隐私保护方面，系统需要妥善处理用户注册信息、学习行为数据和个人资料等敏感内容。用户密码在服务端以哈希形式存储，避免明文保存带来的

泄露风险。学习行为数据仅供用户本人和授权管理员查看，不对外公开或用于未经授权的目的。在系统部署过程中，敏感信息应通过 HTTPS 等加密通道传输，降低数据在传输过程中被截获的风险。虽然本文不涉及专业的安全认证和隐私合规审计，但这些基础措施为系统的数据安全提供了必要的保障。

在国际化与本地化方面，当前系统主要面向中文用户群体，界面语言和内容均以中文呈现。结合本科毕业设计的实现边界，本文当前并未对多语言支持进行具体实现，而是将其作为后续可能的扩展方向保留讨论空间。

第4章 系统设计

4.1 总体架构设计

本系统采用移动客户端、服务端、管理后台端三部分协同工作的总体架构。其中，移动客户端作为学习活动的主要载体，负责课程展示、练习交互、挑战参与、社交反馈与个人成长呈现；服务端负责统一提供认证、课程内容、提交记录、奖励发放、社交关系和进度管理等业务能力；管理后台端负责课程、题目、挑战、用户与运营信息维护。三部分之间以统一的 OpenAPI 契约作为接口约束基础，从而保证数据结构、权限边界与交互流程的一致性。

为便于说明三端在系统中的职责分工，本文将其核心定位归纳如表4-1所示。可以看出，移动客户端承担学习活动入口与交互反馈职能，服务端承担统一业务支撑与数据处理职能，管理后台端承担内容维护与运营配置职能，三者共同构成完整的学习闭环。

为了使三端之间的关系更加直观，论文终稿建议补充一张总体架构图，展示移动客户端、服务端、管理后台端以及 OpenAPI 契约文档之间的连接关系和主要数据流向。

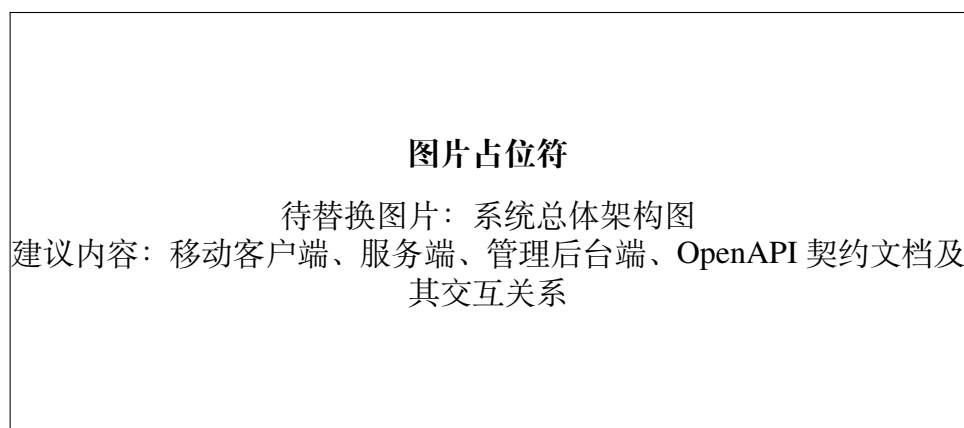


图 4-1 系统总体架构图占位

该架构的核心设计思想不是三端并列展开，而是围绕移动客户端学习闭环进行支撑式组织。也就是说，服务端与管理后台端的存在目的是保证移动客户端学习过

表 4-1 系统三端职责划分

端类别	核心定位	主要职责
移动客户端	学习活动主载体	课程学习、章节阅读、编码练习、挑战参与、社交互动、成长反馈
服务端	统一业务支撑中心	认证鉴权、课程内容提供、提交记录、奖励结算、进度管理、社交数据处理
管理后台端	内容维护与运营支撑	课程管理、题目管理、挑战配置、用户管理、公告与运营配置

程能够持续、稳定、可配置地运行，而不是与移动客户端分别形成独立的论文主体。在这种架构思路下，服务端被设计为统一的数据与业务处理中心，避免移动客户端直接访问数据库或执行复杂业务逻辑，从而保证数据安全性和业务规则的一致性。管理后台端则作为运营人员的操作界面，其所有操作最终都通过服务端接口生效，这种间接访问方式使系统能够在管理后台端和移动客户端之间建立统一的业务规则执行层。

从部署角度看，服务端作为独立服务运行，管理后台端作为静态前端应用部署，移动客户端则作为原生应用分发到用户设备。这种分离部署方式使各端可以独立迭代和发布，减少了版本耦合带来的协调成本。同时，服务端统一对外提供接口，使移动客户端和管理后台端共享同一套业务逻辑和数据模型，降低了多端维护的复杂度。

4.2 功能结构设计

根据移动客户端路由与页面组织，系统移动客户端功能结构可划分为启动与认证、课程学习、编码练习、闯关挑战、每日挑战、社交互动、个人中心七个部分。启动与认证负责完成用户状态识别与基础登录流程；课程学习负责承载课程目录、课程详情与章节内容；编码练习负责连接题目内容、代码提交与结果反馈；闯关挑战与每日挑战用于组织具有目标性的任务链；社交互动负责呈现好友、动态与排行榜；个人中心负责展示用户资料、学习统计和奖励信息。

在启动与认证模块中，系统需要处理应用冷启动时的初始化流程，包括本地状态检查、登录态恢复判断和引导页面展示。如果本地存在可用的认证信息，系统可直接恢复到主流程界面；如果认证信息失效或不存在，则引导用户进入登录或注册流程。这种设计使已登录用户能够较快恢复学习状态，同时保证未登录用户能够顺利进入系统。

在课程学习模块中，功能结构进一步细分为课程列表、课程详情和章节阅读三个层次。课程列表提供全部课程的概览信息和筛选能力，课程详情展示课程介绍、章节目录和相关练习，章节阅读则承载具体的知识内容。这种三层结构使学习者能够从宏观到微观逐步深入，同时保持每一层的信息密度在移动设备可接受范围内。

在编码练习模块中，功能结构围绕题目展示、代码编辑和结果反馈三个核心环节展开。题目展示需要清晰呈现题目要求、示例输入输出和注意事项；代码编辑需要提供基础的输入环境和语法辅助；结果反馈需要及时返回编译结果、运行输出和测试用例通过情况。这三个环节构成了练习功能的完整闭环，任何一环的体验缺陷都会影响学习者的练习积极性。

与之对应，管理后台端支撑结构主要包含课程管理、挑战管理、题目管理、用户管理、内容审核和公告配置等能力。管理后台端并不直接面向学习活动本身，而是通过维护内容与配置规则，为移动客户端持续提供可消费的数据与管理能力。在课程管理中，运营人员可以创建和编辑课程信息、组织章节结构、关联练习题目和控制课程上下线状态。在挑战管理中，运营人员可以配置关卡顺序、设置通关条件、调整奖励规则和管理挑战可用性。在用户管理中，运营人员可以查看用户基础信息、学习统计和行为记录，为运营决策提供数据支持。

在功能结构表达上，建议补充一张移动客户端功能结构图，以更清楚地对应课程学习、编码练习、挑战任务、社交互动与个人中心等模块之间的组织关系。

图片占位符

待替换图片：移动客户端功能结构图

建议内容：启动与认证、课程学习、编码练习、闯关挑战、每日挑战、社交互动、个人中心

图 4-2 移动客户端功能结构图占位

4.3 核心业务流程设计

在核心学习流程上，学习者首先通过认证模块进入系统，随后浏览课程列表并进入具体课程与章节，在完成知识阅读后进入编码练习或挑战任务。练习与挑战的提交结果会驱动学习进度、经验值、奖励和统计数据更新，并进一步影响排行榜、个人中心与社交动态展示。这样的设计使得知识学习、行为记录和激励反馈形成闭环，从而增强学习过程的连续性。

具体而言，学习流程的起点是认证模块。学习者输入账号密码完成登录，或使用注册功能创建新账号。登录成功后，服务端返回认证 Token，移动客户端将其持久化到本地存储，并在后续请求中自动携带。服务端同时提供 Token 刷新接口，客户端当前主要在登录失效时进行统一处理并引导重新登录，从而维持基础会话管理能力。

进入主界面后，学习者首先接触的是课程列表。课程列表展示标题、简介、进度等基础信息，并支持搜索与筛选。学习者选择感兴趣的课程后进入课程详情页，查看课程介绍和章节目录。章节按照顺序排列，已完成的章节标记为完成状态，未完成的章节中第一个未解锁的章节标记为当前学习进度。这种视觉反馈使学习者能够直观了解自己的学习位置和剩余内容。

在章节阅读页面，学习者浏览知识内容，内容以图文形式组织，适应移动设备的阅读习惯。阅读过程中，系统结合章节完成状态与课程进度记录学习过程。完成章节阅读后，学习者可以选择进入关联的练习页面，通过编码实践巩固所学知识。练习页面展示题目要求和代码编辑区，学习者编写代码后提交，服务端对代码进行评测并返回结果反馈。根据反馈，学习者可以修改代码重新提交，逐步完成练习目标。

在每日挑战流程中，系统按照每日任务入口向学习者提供限时或周期性任务，学习者完成提交后更新打卡记录、连续学习状态与额外奖励。该流程与普通挑战相比，更强调节奏控制与日常激励，从而提高用户的持续访问频率。每日挑战的设计考虑了学习者的时间安排习惯，任务难度适中，完成时间可控，使学习者能够在碎片时间内完成并获得成就感。连续打卡机制则通过累积奖励激励学习者保持日常学习习惯，但系统设计时也考虑了断签后的恢复机制，避免因偶然中断造成过度挫败感。

为了帮助读者理解学习活动如何从登录进入课程、练习、挑战再回到成长反馈，建议在此处补充一张核心学习流程图。

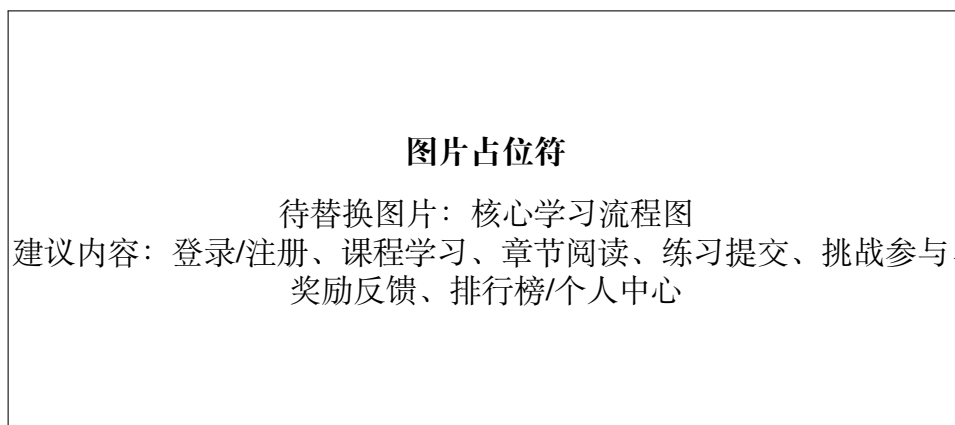


图 4-3 核心学习流程图占位

4.4 接口与数据设计

为了保证多端协同一致性，系统将 OpenAPI 文档作为共享接口真源，对路径、方法、请求体、响应结构、受众标记与权限边界进行统一定义。服务端实现依据该接口文档组织路由与处理逻辑，移动客户端通过统一的请求服务访问接口并完成 Token 注入、异常处理与结果解包，管理后台端则通过统一 API 封装消费管理能力。

在接口设计原则上，系统遵循 RESTful 风格，使用 HTTP 方法表达操作语义，使用路径参数和查询参数传递定位信息，使用请求体传递复杂数据。响应结构采用统一的包络格式，包含状态码、消息文本和数据载荷三个部分，使调用方能够以一致的方式处理成功和失败情况。对于列表类接口，系统支持分页参数，使移动客户端能够按需加载数据，避免一次性传输大量内容。

在数据结构上，系统围绕课程、章节、练习、挑战、每日挑战、用户资料、好友关系、成长奖励与学习进度等对象进行设计。其中，学习者侧的数据对象需要兼顾展示需求与交互需求，而服务端数据结构则更强调持久化与状态更新逻辑。通过契约文档、状态机说明与字段词典的配合，可以在设计阶段约束字段语义与状态流转，降低实现阶段出现理解偏差的可能性。

课程对象包含课程标识、标题、简介、封面图、分类、难度等级、上下线状态和创建时间等字段。章节对象包含章节标识、所属课程、标题、内容、顺序号和关联练习标识等字段。练习对象包含练习标识、题目描述、示例输入输出、测试用例、难度等级和关联章节等字段。挑战对象包含挑战标识、标题、描述、关卡列表、奖励规则和可用性状态等字段。用户资料对象包含用户标识、昵称、头像、等级、经验值、注

册时间和最近活动时间等字段。

在状态设计方面，系统对关键实体定义了明确的状态集合和流转规则。例如，课程具有草稿、已上线和已下线三种状态，状态转换由管理后台端操作触发。挑战参与记录具有未开始、进行中、已完成和已放弃四种状态，状态转换由学习者的操作触发。这些状态定义和流转规则在契约文档中以状态机形式描述，为多端实现提供了一致的理解基础。

在接口与数据设计部分，建议补充一张接口协同或关键数据流图，用于呈现客户端请求、服务端处理以及管理后台端配置变更之间的关系。

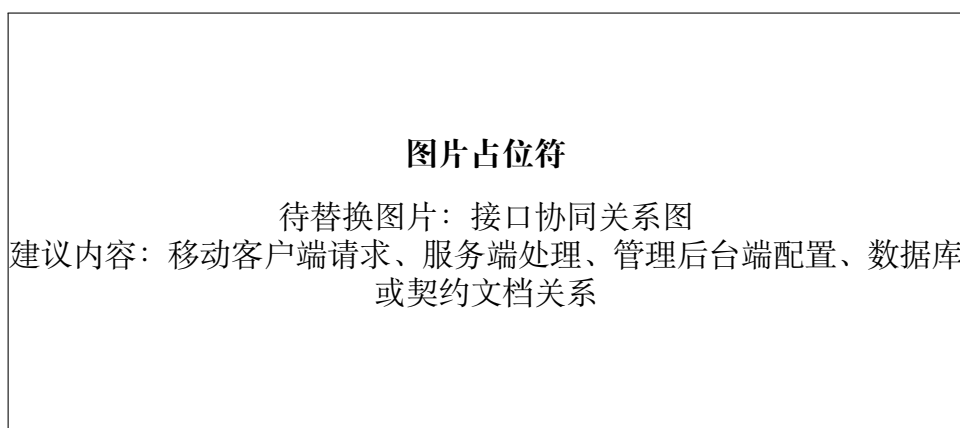


图 4-4 接口协同关系图占位

4.5 服务端与管理后台端协同设计

服务端路由按照学习者侧与管理侧受众进行组织，其中学习者相关接口主要服务于课程、资料、进度、挑战、奖励、社交与提交流程；管理侧接口则服务于课程、挑战、题目、用户、反馈与配置维护。这样的划分方式能够使系统在权限控制与职责边界上更加清晰，也有利于后续联调测试中的问题定位。

学习者侧接口采用统一的认证中间件进行身份校验，从请求头中提取 Token 并解析用户身份，将身份信息附加到请求上下文中供后续处理使用。这种设计使业务处理逻辑无需关心认证细节，只需从上下文中获取已验证的用户身份即可。管理侧接口除了认证校验外，还增加了角色校验，确保只有具备管理权限的用户才能访问管理功能。

在路由组织上，学习者侧接口以学习者视角组织资源路径，例如课程列表、我的进度、我的挑战等；管理侧接口则以管理视角组织资源路径，例如课程管理、用户管理、统计报表等。这种路径命名方式使接口的用途和受众一目了然，减少了调用方的理解成本。

管理后台端通过认证守卫、统一 API 访问层和页面级管理入口完成支撑工作，主要用于内容维护与运营配置。管理后台端主要检查登录状态，未登录用户会被重定向到登录页面；更具体的管理权限约束主要由服务端接口控制。统一 API 访问层封装了所有与服务端的通信逻辑，包括请求发送、错误处理和响应解析，使页面组件只需关注业务逻辑而无需处理网络细节。

由于本文以移动客户端学习应用为主体，因此管理后台端设计在论文中只说明其对内容供给与管理闭环的支撑意义，而不对各个管理页面展开细粒度实现分析。但需要强调的是，管理后台端的完备性直接影响移动客户端的内容质量和运营效率。如果管理后台端缺乏灵活的内容编辑能力，运营人员将难以快速响应内容更新需求；如果管理后台端缺乏有效的数据查看能力，运营决策将缺乏数据支撑。因此，在系统设计时，管理后台端的功能范围与易用性被纳入整体考虑，确保其能够切实支撑移动客户端的持续运营。

在数据流向上，管理后台端的操作通过服务端接口持久化到数据库，移动客户端通过服务端接口读取最新数据。这种单向数据流设计使数据变更的源头唯一且可追溯，避免了多端同时修改数据可能带来的冲突和不一致。在缓存策略上，移动客户端对课程列表等相对静态的数据进行本地缓存，减少重复请求；对进度、排行榜等动态数据则优先从服务端获取，保证信息的时效性。

在安全设计方面，系统从接口访问控制、数据传输保护和输入验证三个层面保障安全性。接口访问控制通过 Token 认证和角色校验实现，确保只有合法用户能够访问其权限范围内的资源。数据传输保护在部署层面可结合 HTTPS 等方式进行实现，以降低敏感信息在传输过程中被截获或篡改的风险。服务端对关键输入进行了基础类型校验和必要的业务校验，减少异常输入导致的数据错误。虽然本文不涉及专业的安全审计和渗透测试，但这些基础安全措施为系统的稳定运行提供了必要的保障。

在错误处理设计方面，系统建立了分层的错误处理机制。服务端对业务错误、认证错误和系统错误进行分类，分别返回对应的 HTTP 状态码和错误信息。移动客户端根据错误类型采取不同的处理策略：对于网络错误，提示用户检查网络连接并提

供重试选项；对于认证错误，引导用户重新登录；对于业务错误，展示服务端返回的具体错误信息。这种分层处理机制使错误信息对用户更加友好，同时也便于开发人员进行问题排查和定位。

在日志与监控设计方面，服务端记录了基础请求日志信息，包括请求路径、响应状态、处理时间和异常详情等。这些日志信息在开发和测试阶段用于问题定位，在运行阶段也可作为观察系统状态的基础。移动客户端当前主要保留基础错误输出与状态提示，用于辅助调试与用户反馈。虽然本文未实现完整的分布式追踪和实时监控体系，但基础的日志记录为系统的可观测性提供了初步支持。

在数据库设计方面，系统采用关系型数据库进行数据持久化，通过合理的表结构设计和索引策略保证基础查询性能。核心数据表包括用户表、课程表、章节表、练习表、挑战表、提交记录表、进度表、好友关系表和奖励记录表等。表之间通过外键关联建立数据一致性约束，同时避免过度规范化导致的查询复杂度上升。在数据库访问层，系统使用连接池管理数据库连接，通过预编译语句防止 SQL 注入攻击，这些基础实践为数据安全和访问性能提供了保障。

第 5 章 系统实现与测试

5.1 移动客户端关键功能实现

在应用入口实现上，系统通过 Flutter 作为运行容器，结合 ScreenUtil 完成不同屏幕尺寸下的界面适配，并在应用初始化阶段依次完成 Firebase、本地存储与全局依赖注入等准备工作。系统主程序通过 GetMaterialApp 统一管理路由、主题、绑定关系与页面切换，同时在应用外层增加离线感知壳层，用于在网络不可用时提示用户学习记录将暂存本地。这种初始化顺序的设计考虑了移动客户端在真实运行环境中的不确定性，将基础服务准备放在用户可见界面之前，有助于减少运行期异常对用户体验的直接影响。

在认证与接口访问实现上，移动客户端通过 ApiService 对 Dio 请求进行统一封装，集中处理基础地址、超时控制、请求头、认证 Token 注入与错误拦截逻辑。当接口返回未授权状态时，系统能够通过统一处理流程回收登录状态，避免页面在失效会话下继续误判为可用。这样的设计降低了页面直接处理网络细节的复杂度，有助于维持 View、Controller 与 Service 之间的职责清晰。ApiService 的封装策略还结合页面层的错误处理分支，对连接超时、接收超时和服务异常等场景给出较明确的提示，使学习者能够在遇到网络问题时获得可理解的反馈，而不是面对技术性错误代码。

在学习进度与本地数据实现上，系统使用 StorageService 对认证信息与本地键值数据进行封装，并在 ProgressService 中维护章节完成、课程进度、挑战通关、奖励发放、每日挑战记录与待同步动作等学习状态。该服务不仅负责本地持久化，还结合网络状态监听在恢复联网时处理本地待同步动作队列，从而提高弱网环境下学习行为记录的连续性。ProgressService 的设计将学习状态分为即时状态与待处理状态两类，即时状态用于界面渲染，待处理状态用于网络恢复后的后续处理，这种分离策略使界面响应与状态管理之间取得平衡。

从页面组织角度看，移动客户端围绕课程、章节、练习、挑战、每日任务、社交、个人中心等场景建立了较完整的学习流程。课程与章节页面负责知识内容承载，练习页面负责题目交互与提交入口，挑战与每日挑战页面负责阶段性目标组织，社交与个人中心页面则承担外部激励与成长反馈展示，形成了从学习到反馈再到激励的

闭环实现。在页面实现层面，系统采用 GetX 的状态管理模式，将页面状态、业务逻辑与界面渲染进行分离，使每个页面的 Controller 只关注该页面特有的数据获取、状态转换和用户交互响应，而 View 层则专注于根据状态变化更新界面元素。

在课程学习功能的具体实现中，课程列表页面以列表方式获取并展示课程数据，同时结合搜索与筛选能力帮助学习者快速定位目标课程。课程详情页面围绕课程封面、课程说明、进度概览与章节列表组织信息，使学习者能够从总体介绍过渡到具体章节。章节阅读页面提供结构化内容展示、章节完成标记与示例代码展示，并结合本地进度记录保持学习过程的连续性。

在编码练习功能的实现中，练习页面需要同时承载题目描述、代码编辑区和结果反馈区。考虑到移动设备屏幕空间有限，系统将工作区划分为代码、预览与运行检查等视图，并通过底部操作栏组织提交与 AI 帮助入口，使学习者在不同操作阶段能够聚焦于当前任务。代码编辑区虽然受限于移动设备输入体验，但系统通过草稿保存、本地预检查与公开测试用例展示等方式降低练习过程中的操作与理解成本。提交后的结果反馈包括评测结果、公开用例情况与辅助提示信息，系统将这些信息以结构化方式呈现，帮助学习者快速定位问题所在。

在挑战功能的实现中，闯关挑战页面按照顺序组织题目序列，学习者通常需要先完成前置挑战才能进入后续内容。这种渐进式解锁机制既保证了学习路径的层次性，也为系统提供了清晰的学习进度追踪点。每日挑战页面则围绕“当天任务”展开，学习者完成后更新对应记录，并与经验值、连续学习天数等成长数据产生联动。这些机制的实现依赖于服务端对挑战状态、参与记录和奖励发放的统一管理，移动客户端则通过接口获取状态并结合本地记录更新展示结果。

在社交功能实现方面，好友页面支持好友列表查看、添加好友请求和好友学习动态浏览。排行榜页面展示学习者排名信息，个人中心页面整合用户资料、学习统计、获得徽章和等级信息，为学习者提供全面的成长概览。这些社交与反馈功能的实现不仅依赖本地数据渲染，还需要与服务端保持适度的状态同步，以确保排行榜和好友动态等信息的时效性。

在实现展示层面，建议优先补充移动客户端关键页面截图，以增强第 5 章的工程真实感和可验证性。首页、课程页、练习页、挑战页和个人中心页是最值得优先准备的界面材料。

图片占位符

待替换图片：移动客户端首页或课程列表截图
建议内容：首页导航、课程列表、搜索筛选、学习进度信息

图 5-1 移动客户端首页与课程列表界面占位

图片占位符

待替换图片：练习页面截图
建议内容：题目描述、代码编辑区、提交按钮、运行结果或错误反馈

图 5-2 编码练习与结果反馈界面占位

图片占位符

待替换图片：挑战页或个人中心截图
建议内容：闯关挑战、每日任务、经验值、徽章、排行榜或成长统计

图 5-3 挑战任务与个人成长界面占位

5.2 服务端与管理后台端关键实现

服务端在实现上采用统一路由树组织方式，将认证、学习者功能与管理侧功能分别挂载在明确的路径层级下。学习者相关接口覆盖课程、资料、好友、动态、排行榜、挑战、每日挑战、奖励、提交与进度等核心能力，能够为移动客户端提供完整的业务支撑。管理侧接口则负责统计、课程、挑战、练习、用户、公告与配置维护，用于保障学习内容与运营数据的持续更新。这种路由组织方式使接口职责在代码结构层面即可辨识，有利于开发过程中的功能定位与问题排查。

在服务启动流程中，服务端依次完成配置加载、数据库连接池创建、迁移执行、种子数据准备、中间件挂载与 OpenAPI 文档导出。这种实现方式使系统在开发与联调阶段能够同时提供业务接口和接口文档访问能力，便于多端协同开发和问题定位。配置加载阶段从环境变量和配置文件读取数据库连接信息、服务监听地址、认证密钥等运行时参数，为后续模块初始化提供基础配置。数据库连接池的创建采用异步方式，在连接建立完成后执行迁移脚本，确保数据库模式与代码期望保持一致。

在业务逻辑实现层面，服务端将请求处理分为参数提取、权限校验、业务执行和响应封装四个阶段。参数提取阶段从请求路径、查询字符串和请求体中获取调用方提供的参数，并进行基础类型校验。权限校验阶段根据接口配置的受众标记判断当前请求是否来自合法来源，并通过 Token 解析获取调用者身份。业务执行阶段调用对应的服务函数完成数据库查询或状态更新操作。响应封装阶段将业务结果统一包装为 ApiResponse 格式返回，对于异常情况则转换为 ApiError 结构并附加适当的 HTTP 状态码。

管理后台端通过统一的 Axios 实例封装请求逻辑，并在请求拦截与响应拦截中处理 Token 注入和未授权重定向，从而为课程管理、挑战管理、用户管理与公告配置等功能提供一致的访问方式。虽然管理后台端不是论文主体，但其实现保证了课程内容与运营数据能够持续服务于移动客户端学习场景。在页面访问控制上，管理后台端至少具备登录态校验与未授权跳转能力，使管理功能能够在统一入口下完成访问约束。

在课程、挑战与用户管理功能的实现中，管理后台端当前主要提供基础管理能力，支持常见字段的创建、编辑、状态调整与删除。这些管理操作直接影响移动客户端中学习者可见的课程、挑战与运营信息。在用户管理功能中，管理后台端还提供

用户列表查看、状态调整和基础统计信息，用于辅助日常维护与运营观察。

考虑到论文需要体现支撑端的可操作性，建议在本节补充一张管理后台端关键页面截图，例如课程管理页、挑战配置页或用户管理页。

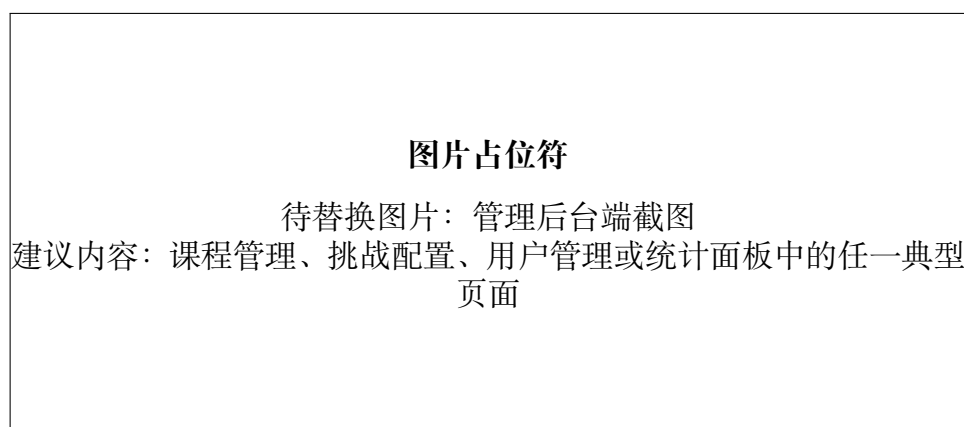


图 5-4 管理后台端关键页面占位

5.3 功能测试与联调验证

本系统测试主要围绕功能验证、契约一致性与关键链路联调展开。当前验证重点包括认证流程、学习者核心业务、管理后台端主要接入点以及若干关键跨端链路。测试策略按契约核对、服务端测试与核心接口调用验证逐步建立信心，其重点不是大规模自动化覆盖率，而是验证核心业务流是否能够稳定闭环。

在契约一致性方面，系统以 OpenAPI 文档为基础核对路径、方法、状态码、成功响应包络、错误结构以及受众与权限边界是否符合定义。该部分验证的价值在于防止接口形状漂移，从而降低移动客户端和管理后台端在消费接口时出现理解偏差的风险。相关过程主要结合契约核对、服务端测试与核心接口调用验证展开，以确认正常请求能够返回预期结构，异常请求能够返回相对一致的错误响应。

在联调验证方面，移动客户端重点关注注册登录、课程访问、练习提交、挑战参与、每日任务、个人资料、奖励信息与排行榜等高价值接口调用链路。注册登录链路测试覆盖正常注册、重复注册提示、密码错误提示、Token 获取和 Token 刷新等场景。课程访问链路测试覆盖课程列表加载、课程详情查看、章节内容获取和阅读进度保存等场景。练习提交链路测试覆盖代码提交、结果获取、错误反馈和多次提交

等场景。挑战参与链路测试覆盖关卡列表获取、题目详情查看、答案提交和通关状态更新等场景。

管理后台端则重点验证登录、课程维护、挑战维护、用户管理、统计与公告配置等真实接入点。课程维护测试覆盖课程创建、编辑、上下线操作后移动客户端是否能够正确反映状态变化。挑战维护测试覆盖关卡配置调整后移动客户端的挑战流程是否按新规则执行。用户管理测试覆盖用户状态变更后相关权限和数据的正确性。统计功能测试验证基础数据汇总和展示是否符合预期。

跨端验证则重点关注管理后台端创建内容后移动客户端可见、学习者行为产生后服务端状态正确更新、Token 失效后多端进入正确失效处理流程等关键场景。在内容同步场景中，测试用例包括管理后台端新建课程后移动客户端课程列表是否出现新课程、管理后台端修改挑战奖励后移动客户端挑战完成时是否按新规则发放奖励。在状态更新场景中，测试用例包括学习者完成练习后个人中心统计是否及时更新、学习者完成挑战后排行榜排名是否正确变化。在会话管理场景中，测试用例包括 Token 过期后移动客户端是否引导重新登录、管理后台端会话超时后是否跳转至登录页面。

如果需要进一步增强测试说服力，建议补充一张联调验证截图，例如接口文档页面、接口调试结果页或跨端联调记录页，用于支撑契约一致性和功能闭环验证。

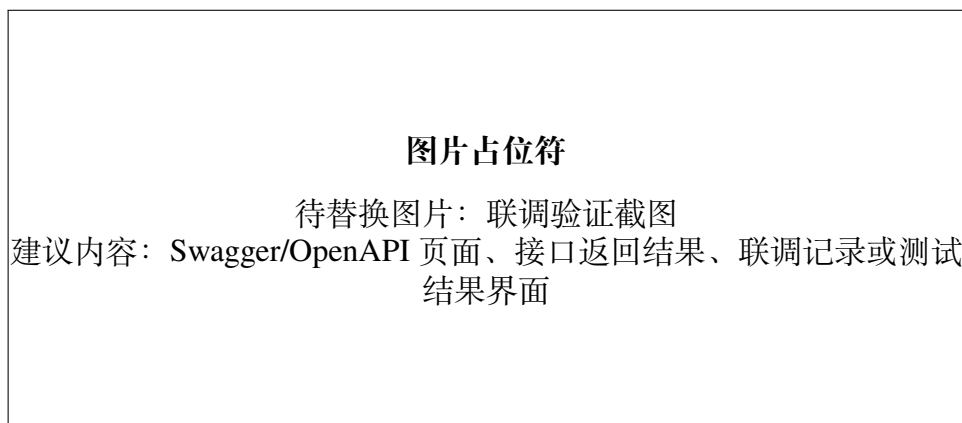


图 5-5 联调验证与接口测试截图占位

5.4 测试结果分析

从测试目标来看，本系统当前更强调功能正确性、接口一致性与多端协同可用性，而不是性能压测或教学效果实验。因此，论文中的测试结论主要用于说明系统在既定实现范围内能够完成核心学习闭环，相关接口在契约约束下具备基本一致性，三端之间能够完成主要业务流程协作。

在功能正确性方面，相关验证说明移动客户端能够围绕注册登录、课程学习、编码练习、挑战参与、每日任务和社交互动形成较完整的主流程。各功能模块在正常使用路径下能够返回预期结果，异常情况也能够给出相对明确的提示信息。在接口一致性方面，相关验证说明服务端接口的实现与 OpenAPI 文档定义基本保持一致，主要端点的请求参数、响应结构和错误处理符合契约约定。在三端协同方面，相关验证说明管理后台端的内容维护能够反映到移动客户端展示侧，学习者行为产生后相关状态也能够与服务端与移动客户端之间完成基本同步。

同时也应看到，移动学习场景对真实网络环境、内容规模增长和长期使用数据仍然较为敏感，后续如需进一步提升论文深度或系统成熟度，可在更多真实运行数据积累后补充性能分析、用户研究或长期学习效果评估。但这些内容不属于本文当前实现与验证的核心范围。具体而言，当前测试主要在局域网环境和模拟数据条件下进行，尚未在真实弱网环境、大规模并发访问和长期用户行为积累等条件下进行充分验证。此外，游戏化机制对学习动机和持续学习行为的实际影响需要更长时间的用户跟踪研究才能得出可靠结论，本文的实现与测试工作为这类后续研究提供了系统基础，但本身并不声称已经验证了游戏化学习效果的因果关系。

在已知限制方面，移动客户端的代码编辑体验受限于移动设备屏幕尺寸和虚拟键盘输入效率，虽然系统通过快捷符号和基础高亮进行了优化，但与桌面开发环境相比仍存在明显差距。AI 辅助功能目前仅提供错误解释和提示，尚未实现个性化的学习路径推荐或自适应难度调整。社交功能中的好友互动和动态分享相对基础，尚未支持实时消息或协作学习等更复杂的社交场景。这些限制既来自当前实现范围的界定，也反映了移动学习应用在设计时需要面对的真实约束，在后续版本中可根据实际使用反馈逐步优化和扩展。

从测试方法论角度反思，本系统的测试策略侧重于功能验证和联调确认，采用了从契约层到跨端层的分层测试思路。这种策略的优势在于能够快速发现接口不匹

配和功能缺陷，测试成本相对较低，适合开发迭代阶段使用。但其局限在于对边界条件和异常路径的覆盖不够全面，对性能、安全和兼容性等方面的验证较为有限。如果后续需要提升系统质量，可以考虑引入自动化单元测试、接口契约测试和基础性能基准测试，形成更加完整的测试体系。不过，这些扩展工作需要在开发流程和团队技能方面进行相应投入，其实施优先级应结合实际项目需求和资源约束综合评估。

在测试环境方面，当前测试主要基于开发环境和模拟数据进行，尚未构建独立的测试环境或使用生产级别的数据规模。这意味着某些与数据量相关的潜在问题可能在当前测试阶段无法暴露，例如课程数量增多后的列表加载性能、用户数量增长后的排行榜计算效率等。这些问题在系统初期可能不会显现，但随着使用规模扩大可能需要关注。本文在测试部分如实记录了这一局限，并建议在后续工作中根据实际部署情况补充相关测试。

在测试整理方面，测试过程中的关键结论和发现已在正文中进行归纳总结，以支持对系统实现质量的客观评估。这种总结方式有助于后续回归测试和问题追溯，但并不意味着本文已经建立了覆盖所有场景的完整测试文档体系。

从工程实现角度看，上述验证工作虽然以核心链路为主，但已经能够为论文中的系统实现分析提供较稳定的事实依据。一方面，移动客户端的课程学习、章节完成、练习提交、挑战参与与社交展示形成了可连续观察的主流程；另一方面，服务端接口、管理后台端维护操作与契约文档之间也建立了较明确的对应关系。这意味着论文在讨论系统设计与实现时，并不是停留在抽象方案层面，而是建立在已有代码、接口与基础验证结果之上的总结性描述。

同时，这种以核心链路为中心的验证方式也符合本科毕业设计的实现边界。对于当前版本而言，论文更关注“系统是否能够围绕前端学习场景形成基本闭环”，而不是追求覆盖所有边界条件的工业级验证深度。因此，在有限开发周期内优先验证课程、练习、挑战、奖励与多端协同等关键链路，能够更直接地支撑本文关于系统可实现性与工程完整性的讨论。

在持续集成与部署实践方面，虽然本文未将 CI/CD 流程作为研究重点，但在开发过程中已建立了基础的多端协作工作流。移动客户端通过 Flutter 的热重载特性实现快速迭代，服务端通过 Cargo 的构建系统保证编译一致性，管理后台端通过 npm 脚本管理开发和构建流程。三端之间的接口变更通过 OpenAPI 文档进行同步，文档更新后各方依据最新定义调整实现。这种轻量级的协作方式在团队规模较小时能够

有效运转，但在规模扩大或发布频率提高后，可能需要引入更正式的版本管理和自动化测试流程。

在代码质量保障方面，系统开发过程中遵循了基础的代码规范和质量检查实践。移动客户端代码遵循 Dart 语言的有效风格指南，服务端代码通过 Clippy 进行静态检查，管理后台端代码通过 ESLint 进行规范校验。这些工具能够在编译或提交阶段发现常见的代码问题，减少低级错误进入测试环节的概率。虽然这些实践不能替代系统性的代码审查和架构评估，但作为基础质量保障手段，它们为代码可维护性提供了初步支持。

结 论

本文围绕游戏化前端学习移动应用的设计与实现展开研究，完成了系统需求梳理、总体架构设计、关键功能实现与功能验证分析。论文以移动客户端为主体，从课程学习、编码练习、闯关挑战、每日挑战、社交互动和个人成长反馈等方面构建了学习闭环，并结合服务端、管理后台端与 OpenAPI 契约约束实现多端协同。

在系统实现层面，本文形成了一套较清晰的移动客户端优先实现方案。移动客户端通过 Flutter、GetX、Dio 与 GetStorage 组合实现跨平台界面、状态管理、接口访问与本地持久化；服务端通过 Rust、Salvo 与 PostgreSQL 提供统一业务能力；管理后台端负责课程、挑战、用户和运营信息维护；OpenAPI 文档则用于统一接口语义与联调边界。相关设计能够支持系统在当前范围内完成主要业务流程。

在验证层面，系统已完成注册登录、课程访问、练习提交、挑战参与、奖励反馈、排行榜展示以及三端基础联调等核心流程的功能验证，这说明其在当前实现范围内具备较明确的工程可行性。

同时，本文仍存在一定不足。例如，系统测试主要集中于功能验证与联调验证，尚未结合长期用户数据开展学习效果评估；移动客户端在更复杂网络环境和更大规模数据场景下的表现仍有进一步观察空间；游戏化机制目前以积分、排行榜和任务节奏为主，后续仍可在个性化激励与更丰富的学习反馈方面继续完善。

后续工作可从三个方向展开：一是继续完善前端学习内容与挑战体系，增强课程覆盖度；二是结合真实用户数据补充可用性与学习持续性分析；三是在保持论文当前工程边界的基础上，进一步优化多端协同和交互体验，以提升系统整体成熟度。

参考文献

- [1] Deterding S, Dixon D, Khaled R, et al. From game design elements to gamefulness: Defining gamification[C]//Proceedings of the 15th International Academic MindTrek Conference. ACM, 2011: 9-15.
- [2] Hamari J, Koivisto J, Sarsa H. Does gamification work? – A literature review of empirical studies on gamification[C]//2014 47th Hawaii International Conference on System Sciences (HICSS). 2014: 3025-3034.
- [3] Ryan R M, Deci E L. Self-determination theory and the facilitation of intrinsic motivation, social development, and well-being[J]. American Psychologist, 2000, 55(1): 68-78.
- [4] 梁迎丽, 刘陈. 人工智能教育应用的现状分析、典型特征与发展趋势[J]. 中国电化教育, 2018(03): 24-30.
- [5] 焦建利. ChatGPT 助推学校教育数字化转型——人工智能时代学什么与怎么教[J]. 中国远程教育, 2023, 43(04): 16-23.
- [6] Sommerville I. Software Engineering[M]. 10th ed. Boston: Pearson, 2015.
- [7] Sylvester T. Designing Games: A Guide to Engineering Experiences[M]. Sebastopol: O'Reilly Media, 2013.
- [8] Kalmpourtzis G. Educational Game Design Fundamentals: A Journey to Creating Intrinsically Motivating Learning Experiences[M]. Boca Raton: CRC Press, 2018.
- [9] Koedinger K R, Anderson J R, Hadley W H, et al. Intelligent tutoring goes to school in the big city [J]. International Journal of Artificial Intelligence in Education, 1997, 8: 30-43.
- [10] Woolf B P. Building Intelligent Interactive Tutors: Student-centered strategies for revolutionizing e-learning[M]. Burlington: Morgan Kaufmann, 2010.
- [11] Brusilovsky P, Millán E. User models for adaptive hypermedia and adaptive educational systems [G]//The Adaptive Web. Berlin: Springer, 2007: 3-53.
- [12] Papamitsiou Z, Economides A. Learning analytics and educational data mining in practice: A systematic literature review of empirical evidence[J]. Educational Technology & Society, 2014, 17(4): 49-64.
- [13] Robins A, Rountree J, Rountree N. Learning and teaching programming: A review and discussion [J]. Computer Science Education, 2003, 13(2): 137-172.
- [14] Heines J M. Teaching web development with Web-based tools[J]. Journal of Computing Sciences in Colleges, 2001, 16(3): 138-146.
- [15] Guo P J. Online Python Tutor: Embeddable web-based program visualization for CS education[C] //Proceeding of the 44th ACM Technical Symposium on Computer Science Education (SIGCSE). ACM, 2013: 579-584.
- [16] Campos S A E, Morales B A M, Núñez Á A V. Open-Source Game Engine & Framework for 2D Game Development[C]//2022 IEEE Engineering International Research Conference (EIRCON). Lima, Peru: IEEE, 2022: 1-4.

附 录

附录 A AI 三级智能提示设计与 Prompt 示例

从论文整体结构来看，附录真正有必要保留的内容，应当是正文中提到但不适合在主文中过度展开、同时又能直接体现系统设计思路的补充材料。对于本文而言，AI 辅助功能虽然不是论文主体，但它与编码练习场景中的学习支持体验密切相关，且正文已经明确说明该功能被限定在“错误解释与提示”范围内。因此，将 AI 辅助提示设计单独作为附录保留是有必要的，它能够补充说明系统如何在不越过“直接给答案”边界的前提下，为学习者提供分层次的帮助。

需要先说明的是，当前契约与示例材料能够直接证明的事实包括：系统存在 `/learner/ai/help` 接口；该接口面向 learner 侧开放；请求中可携带 `exercise_id`、`submission_id`、`request_type`、`source_code` 与 `error_context_json`；响应中可返回 `response_text`、`response_structured_json`、`provider_name`、`token_usage`、`latency_ms` 和处理状态等字段。契约中已经定义了 `request_type` 至少支持 `error_explanation` 与 `hint` 两类请求；mock 数据中还出现了 `hint_level = guided` 这一结构化字段。因此，附录中所谓“三级智能提示”，应理解为本文在既有 AI 辅助能力之上的提示分级设计方案，用于展示本系统在学习支持上的可解释设计，而不表述为契约中已经枚举出三种固定 API 类型。

结合编码练习的学习场景，本文将 AI 辅助提示划分为三级：第一级是**错误定位提示**，目标是帮助学习者理解“哪里出了问题”；第二级是**修正方向提示**，目标是帮助学习者理解“应该往哪个方向改”；第三级是**操作建议提示**，目标是帮助学习者在直接获得完整答案的前提下，得到可以立即尝试的下一步操作。这样的三级结构与移动学习场景相适配：初学者在手机端练习时，最常见的问题并不是完全不会，而是不知道错误产生的位置、原因和第一步该怎么改。

表A-1 对三级提示的定位进行归纳。

在具体实现思路上，三级提示并不是三套完全独立的接口，而更适合作为同一 AI 辅助能力下的三层输出策略。也就是说，移动客户端在调用 `/learner/ai/help` 时，仍然可以沿用统一的请求结构；服务端或 AI 编排层根据当前的 `request_type`、失败上下文、提交记录和提示次数，决定当前应返回更偏向“解释”、更偏向“方向”，

表 A-1 AI 三级智能提示设计

提示级别	设计目标	适用场景	输出边界
一级提示： 错误定位	帮助学习者识别失败位置与现象	刚提交后看到错误、公开用例未通过、布局结果异常	说明错误位置、失败现象与相关字段，不直接给出完整修复代码
二级提示： 修正方向	帮助学习者理解问题原因与可能的调整方向	已知道错误点，但仍不清楚应该修改样式、结构还是断点条件	给出原因判断和调整方向，可引用媒体查询、布局属性、选择器覆盖等概念，但不直接替换整段代码
三级提示： 操作建议	帮助学习者执行下一步尝试并验证修改效果	学习者多次尝试后仍卡住，需要更具体的步骤性帮助	给出 2-4 条可执行操作建议，如“先检查断点条件”“再重写 <code>grid-template-columns</code> ”，但避免直接输出最终答案

还是更偏向“操作建议”的内容。这样既符合契约优先的接口设计思路，也便于后续在不修改主要接口形状的前提下扩展提示策略。

表A-2 对 AI 提示生成时可利用的输入信息进行归纳。这些输入项均可以从当前契约、示例或页面上下文中获得，不需要依赖额外的隐式状态。

表 A-2 AI 提示生成的主要输入信息

输入项	典型来源	作用说明
题目要求	练习详情中的 <code>prompt</code> 字段	说明当前练习的目标，例如实现响应式三列卡片布局
学习者提交代码	<code>source_code</code>	作为 AI 分析错误与给出提示的主要上下文
提交记录信息	<code>submission_id</code> 、提交详情与评测状态	帮助区分是首次错误还是多次重复失败
错误上下文	<code>error_context_json</code>	描述失败用例、错误摘要、断点场景或实际输出
请求类型	<code>request_type</code>	决定当前是偏向错误解释还是偏向提示引导
提示层级	AI 编排规则或结构化返回字段	决定当前输出一级、二级还是三级提示风格

基于上述输入，本文给出一组适用于论文展示的 Prompt 模板示例。这里的 Prompt 内容用于说明提示设计逻辑，而不是声明论文工程中已将模板逐字固化到代码仓库中。附录的目的在于把设计思路展示清楚，使读者能够理解系统为何能够提供

“解释而非代做”的 AI 辅助能力。

一级提示 Prompt 模板示例：错误定位

你是一名面向前端初学者的学习辅助教练。请根据题目要求、学习者提交代码和失败上下文，**只解释当前最可能出错的位置与现象**。

不要直接给出完整答案代码。

请使用简洁中文回答，包含以下三部分：

- 1) 当前错误最可能出现在哪一段结构或样式；
- 2) 它导致了什么可观察现象；
- 3) 学习者下一步应该先检查什么。

这一层提示适合第一次失败或错误现象较明显的场景。例如，当前示例中的 AI 帮助请求给出了 `failing_case = mobile_single_column` 和 `actual_columns = 2`。在这种情况下，一级提示不必解释完整的 CSS 布局原理，只需指出“窄屏断点下仍保留了两列布局，优先检查媒体查询是否生效，以及对应容器是否仍在原有列定义”即可。

二级提示 Prompt 模板示例：修正方向

你是一名面向前端初学者的学习辅助教练。请在不直接给出完整答案的前提下，**解释问题产生的原因，并给出修正方向**。

请结合题目要求、提交代码和失败上下文，输出以下内容：

- 1) 失败最可能由哪一类原因导致；
- 2) 学习者应优先修改布局结构、选择器覆盖、断点条件还是属性值；
- 3) 给出 2 条不含完整答案的方向性建议。

二级提示比一级提示更进一步，它不只告诉学习者“哪里错了”，还告诉学习者“为什么会这样”。例如在响应式卡片布局练习中，如果移动端仍然显示两列，那么二级提示可以指出：问题通常来自媒体查询断点未命中、媒体查询中未覆盖原有布局属性，或者修改了错误的选择器。这样的输出能够帮助学习者从单次报错中形成更稳定的前端调试思路。

三级提示 Prompt 模板示例：操作建议

你是一名面向前端初学者的学习辅助教练。学习者已经获得基础解释，但仍然没有修复问题。请输出**可执行的下一步操作建议**，帮助其继续尝试。

要求：

- 1) 仅给出步骤，不直接生成完整答案；
- 2) 每一步都应能在当前编辑器中立即尝试；
- 3) 最多给出 4 步；
- 4) 最后提醒学习者重新提交并观察哪个公开用例发生变化。

三级提示适合学习者在多次尝试后仍然卡住的情况。它最接近“教学支架”，但仍然不能越过论文中设定的边界。以当前示例为例，三级提示可以设计成：先检查媒体查询断点是否与测试场景一致；再确认媒体查询内是否重写了容器的列定义；接着在本地预览中缩小视口观察列数变化；最后重新提交并关注公开测试是否通过。这样的设计既能提高学习者继续尝试的成功率，也能避免 AI 直接替代学习者完成编码任务。

综合来看，保留 AI 提示设计附录是有必要的，原因不在于该功能本身是否庞大，而在于它能够直接回应论文中的一个关键设计判断：系统的 AI 辅助能力不是面向“自动写代码”，而是面向“在练习受阻时提供分层提示”。这一点若只在正文中概括描述，容易显得抽象；通过附录列出提示层级与 Prompt 模板，能够使该设计更加具体、可解释，也更符合毕业论文附录“补充主文论述而不打断正文节奏”的用途。

附录 B OpenAPI 接口列表与对应功能表单

除了 AI 提示设计外，另一类有保留价值的附录是“OpenAPI 接口列表与对应功能表单”。这类附录的必要性在于：论文正文已经多次强调系统采用契约优先设计，且移动客户端、服务端与管理后台端围绕同一份 OpenAPI 文档协作；如果附录能够进一步把“接口”和“页面/表单”一一对应起来，就能更直接说明契约并非抽象说明书，而是具体指导了登录、资料维护、练习提交、AI 辅助以及后台内容维护等页面功能的字段设计。

在本论文范围内，没有必要把全部 OpenAPI 路径都列为附录。更合适的做法，是只选择那些既与正文高频功能相关、又能代表不同页面表单类型的接口：例如登录

表单、个人资料表单、练习提交表单、AI 辅助请求表单、课程创建表单、挑战创建表单和练习创建表单。这样既能体现契约优先的工程方法，也不会把附录写成冗长的接口手册。

表B-3 对关键接口与对应表单进行归纳。

表 B-3 关键 OpenAPI 接口与对应功能表单

接口路径	方法	对应页面或表单	主要表单字段
/auth/learner/login	POST	学习者登录表单	email、password、device_id、device_name、platform
/learner/profile	PATCH	学习者个人资料与偏好表单	nickname、avatar_url、bio、theme_mode、daily_goal_minutes
/learner/submissions	POST	练习提交表单	exercise_id、source_code、content_version、rule_version
/learner/ai/help	POST	AI 辅助请求表单	exercise_id、submission_id、request_type、source_code、error_context_json
/admin/courses	POST	管理后台端课程创建表单	course_code、title、summary、description、cover_image_url、difficulty、estimated_minutes、status、sort_order、content_version、published_at
/admin/challenges	POST	管理后台端挑战创建表单	challenge_code、title、summary、related_course_id、difficulty、reward_xp、status、sort_order、content_version、stages
/admin/exercises	POST	管理后台端练习创建表单	chapter_id、exercise_code、title、prompt、exercise_type、starter_code、language、difficulty、pass_score、max_attempts_per_day、status、content_version、options、test_cases

为了进一步说明这些接口如何映射到真实页面表单，表B-4 从“功能意图”角度进行补充。

从这些表单可以看出，OpenAPI 对系统的作用不仅是给出一组接口路径，更是提前定义了页面表单应当如何组织字段。以学习者登录表单为例，示例中除 email 和 password 外，还要求提交 device_id、device_name 和 platform。这说明移动客

表 B-4 功能表单的作用与表单意图说明

表单名称	主要作用	设计说明
学习者登录表单	建立认证会话并绑定移动端设备上下文	除账号密码外，还包含设备标识、设备名称与平台信息，说明登录不仅是身份校验，还服务于会话管理与设备侧上下文识别
学习者个人资料表单	维护昵称、头像、简介与学习偏好	字段较少但直接影响个人中心展示和个性化偏好，是移动客户端中典型的轻量编辑表单
练习提交表单	将当前代码与版本信息提交到评测流程	不只是提交源码，还要附带内容版本与规则版本，以保证评测结果与当前练习配置一致
AI 辅助请求表单	把当前卡点上下文发送给 AI 辅助能力	同时包含题目、提交和错误上下文信息，体现 AI 帮助不是孤立问答，而是依附于当前练习状态的上下文文化提示请求
课程创建表单	由管理后台端创建课程基础信息	既包含标题、摘要、封面等展示字段，也包含业务编码、发布状态和版本号等后台维护字段
挑战创建表单	由管理后台端配置挑战与关卡结构	除基础信息外，还包含关卡数组、奖励值和解锁规则，体现挑战表单比普通内容表单更强调状态链与阶段结构
练习创建表单	由管理后台端配置练习题干、选项和测试用例	表单最复杂，既要配置题目本体，也要配置公开/隐藏测试用例与通过阈值，是契约优先设计最能体现结构价值的典型场景

户端登录页在设计时就需要考虑设备上下文，而不是只做最简的账号密码输入。又如个人资料更新接口中出现 `theme_mode` 与 `daily_goal_minutes` 字段，意味着个人资料页并不只是静态信息展示，还承担了偏好设置入口的作用。

在练习与 AI 辅助场景中，契约与表单之间的对应关系更为明显。练习提交表单要求提交 `exercise_id`、`source_code`、`content_version` 和 `rule_version`，这意味着练习页面不仅要保存当前代码内容，还要与当前题目版本和评测规则版本保持一致。AI 辅助请求表单则进一步扩展了上下文维度，除练习标识与提交标识外，还需要提供 `request_type` 和 `error_context_json`。这说明 AI 帮助入口并不是一个脱离练习状态的独立聊天框，而是紧密依附于“当前哪道题、哪次提交、当前报什么错”的上下文辅助入口。

在管理后台端，课程、挑战和练习创建表单则体现了契约优先设计的另一层意义，即通过结构化字段约束后台内容维护行为。课程创建表单需要填写标题、摘要、描述、封面、难度、状态和发布时间等基础字段；挑战创建表单进一步引入 `stages`、星级规则和解锁规则；练习创建表单则最为复杂，既要配置题目描述与起始代码，又要配置选项和测试用例。若没有 OpenAPI 与示例文件约束，这些复杂表单极易在字段命名、数据层级或必填规则上出现前后不一致；而附录把接口与表单对应后，读者可以更直观看出契约如何实质性地参与了后台页面设计。

因此，从论文附录的必要性来看，保留“OpenAPI 接口列表与对应功能表单”是合理的，因为它能够把正文中关于契约优先设计的论述落到具体页面和字段层面。相反，若附录继续保留大而泛的跨端模块映射、测试矩阵和联调场景，则容易与正文已有内容重复，且不如这类“接口-表单”对应关系更直接、更有辨识度。按照当前论文的重点，附录保留 AI 提示设计和 OpenAPI/表单对应这两类内容，已经足以形成清晰、必要且有区分度的补充材料。

致 谢

值此论文完成之际，谨向在本课题研究与论文撰写过程中给予我帮助和支持的老师、同学与家人表示诚挚感谢。

首先，感谢指导教师在论文选题、结构安排与内容修改过程中给予的耐心指导，使我能够逐步明确论文方向并不断完善写作内容。其次，感谢项目开发与学习过程中给予我建议和帮助的同学，使我在系统实现与问题分析阶段获得了许多有益启发。

同时，感谢家人在整个毕业设计阶段给予我的理解与支持，使我能够持续投入到课题研究、系统实现与论文撰写之中。

最后，感谢所有为我的学习与成长提供帮助的人。正是在这些支持和鼓励下，我才能顺利完成本次毕业设计论文。